

Dynamic Queue Monitor

Multi-Channel Dynamic Queue Monitoring System

Muhammet Eray Akbař

Email: erayakbas15@gmail.com

Istanbul Technical University

September , 2026

Abstract

This report presents the design and implementation of a multi-channel dynamic queue monitoring system built with Spring Boot and React. The system models a producer-consumer scenario in which multiple Sender and Receiver threads communicate through a shared bounded queue. Senders attempt to produce one item per second, while Receivers attempt to consume one item per second. When the queue is full, Senders wait; when the queue is empty, Receivers wait.

The implementation focuses on explicit concurrency control, observable worker states, dynamic thread lifecycle management, and a simple monitoring interface. The bounded queue is implemented with `ArrayDeque`, `ReentrantLock`, and `Condition` objects instead of a ready-made `BlockingQueue`, allowing queue waiting and lock contention to be represented directly. The backend collects queue and thread metrics every second, while the React frontend polls the latest snapshot at the same interval. The final system includes dynamic worker creation, group and system stop operations, thread-priority management, backend and frontend tests, setup documentation, and an OpenAPI 3.0 definition.

Contents

1. Introduction	2
2. Implementation / Setup / Methods	2
3. Results	6
4. Discussion	8
5. Conclusion	10

1. Introduction

The purpose of this project is to implement and visualize a concurrent producer-consumer system with multiple Sender and Receiver threads. All workers communicate through a single bounded queue whose capacity is selected by the user. The application must also expose the current queue occupancy and summarize worker states so that the behavior of the system can be observed while it is running.

The main functional requirements addressed by the implementation are:

- Configurable Sender and Receiver counts and queue capacity.
- One production or consumption attempt per worker approximately every second.
- A bounded, thread-safe shared queue.
- Observable `RUNNABLE`, `WAITING`, `BLOCKED`, and `TERMINATED` worker states.
- Dynamic worker addition and group-level or system-level stopping.
- Queue item count and occupancy percentage visualization.
- Backend metrics collected every second and displayed by a React interface.
- REST endpoints for lifecycle operations and status queries.
- Automated backend and frontend tests.

The implementation intentionally keeps the overall architecture compact. No database, message broker, or external state-management library is introduced because the core problem is in-process synchronization and lifecycle monitoring rather than persistence or distributed messaging.

2. Implementation / Setup / Methods

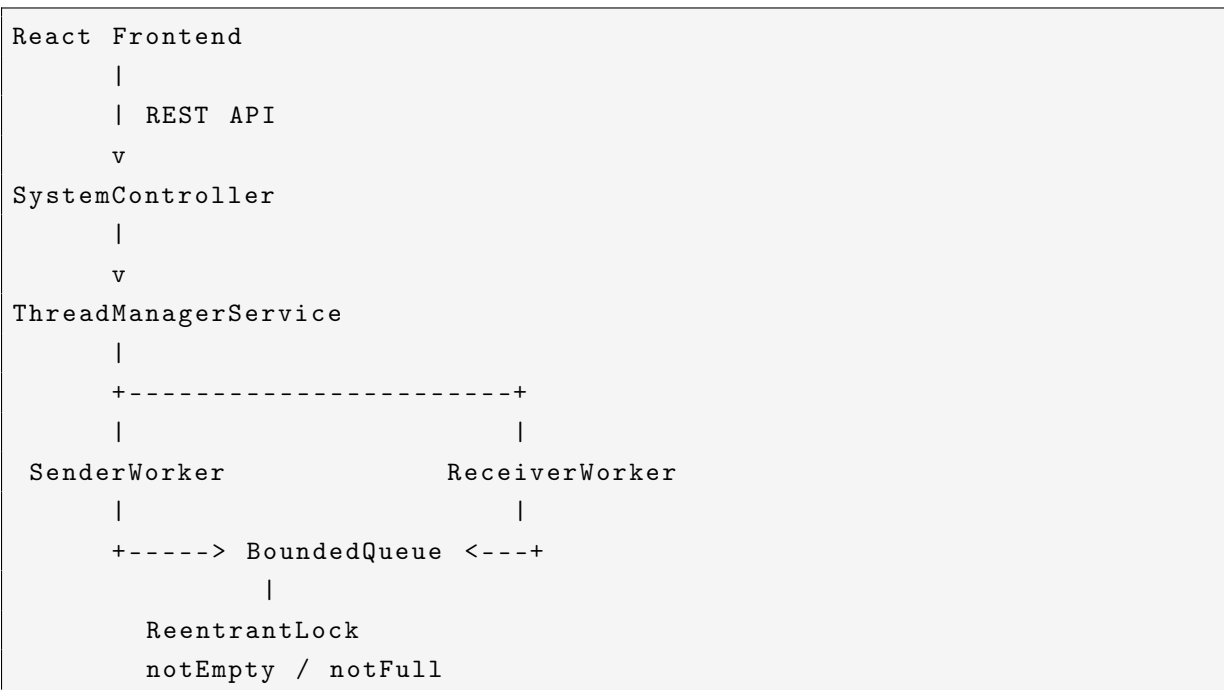
2.1 System Architecture

The project consists of a Spring Boot backend and a React frontend. The frontend communicates with the backend through a REST API. The backend is organized around a small Controller–Service–Worker structure.

1. **SystemController:** Exposes REST endpoints for starting and stopping the system, adding workers, stopping worker groups, changing thread priority, and reading system status.

2. **ThreadManagerService:** Owns Sender and Receiver workers, manages their life-cycle, creates periodic metric snapshots, and coordinates access to the shared queue.
3. **SenderWorker / ReceiverWorker:** Implement the producer and consumer behaviors.
4. **BoundedQueue:** Provides the shared thread-safe data structure and controls full/empty waiting behavior.
5. **React Frontend:** Provides the system controls and visualizes queue and worker-state information.

The high-level data flow is:



All runtime state is kept in memory. This is appropriate for the case because the queue and workers represent a transient concurrent simulation rather than persistent business data.

2.2 Bounded Queue and Synchronization

The shared `BoundedQueue<T>` stores items in an `ArrayDeque`. Access to the queue is protected by a `ReentrantLock`. Two separate `Condition` objects are created from the same lock:

- `notEmpty`: used by Receivers waiting for data.
- `notFull`: used by Senders waiting for available queue capacity.

The `put()` operation first attempts to acquire the lock. If the queue is already full, the Sender changes to the logical `WAITING` state and calls `notFull.await()`. After an item is inserted, one waiting Receiver is signaled through `notEmpty.signal()`.

Similarly, the `take()` operation waits on `notEmpty` while the queue is empty. After removing an item, it signals `notFull` so that a waiting Sender can continue.

The full and empty conditions are checked with `while` loops rather than `if` statements. Therefore, after a worker is awakened it checks the shared condition again before continuing. This is important because another thread may have changed the queue before the awakened worker reacquires the lock.

2.3 Worker Lifecycle and State Model

Common lifecycle behavior is implemented in the abstract `AbstractWorker` class. The `run()` method is final, while the concrete work performed during each cycle is delegated to the abstract `doWork()` method. `SenderWorker` and `ReceiverWorker` provide different implementations of this operation while sharing the same lifecycle logic.

A Sender creates a payload such as `sender-1-data-1` and inserts it into the queue. A Receiver removes one item from the queue. After a successful operation, each worker waits approximately one second before starting its next cycle.

The application uses four logical worker states:

- **RUNNABLE:** the worker is active and able to perform its cycle.
- **WAITING:** the worker is waiting because the queue is full or empty.
- **BLOCKED:** the worker could not immediately acquire the queue lock.
- **TERMINATED:** the worker has been stopped and its loop has finished.

These states are application-level monitoring states. In particular, `BLOCKED` should not be interpreted as a direct mapping to `java.lang.Thread.State.BLOCKED`. Since `ReentrantLock` uses its own synchronizer implementation, the logical state is used to describe the behavior relevant to this application.

The worker state and running flag are declared `volatile` so that updates performed by worker threads are visible to HTTP request and metrics threads. Worker collections use `ConcurrentHashMap`, while `AtomicInteger` counters generate deterministic IDs such as `sender-1` and `receiver-1`.

2.4 Backend Monitoring and REST API

Spring scheduling is enabled in the backend. `ThreadManagerService` creates a new `SystemStatusResponse` snapshot every 1000 ms. Each snapshot contains:

- Current queue size.
- Queue capacity.
- Queue occupancy percentage.
- Sender state counts.
- Receiver state counts.

The REST API is summarized in Table 1.

Table 1: REST API Endpoints

Method	Endpoint	Purpose
POST	/api/system/start	Start or reset the system with selected worker counts and queue capacity.
POST	/api/system/stop	Stop all Sender and Receiver workers.
POST	/api/threads/add	Add Sender or Receiver workers dynamically.
POST	/api/threads/{type}/stop	Stop all workers belonging to one selected type.
PATCH	/api/threads/{type}/{id}/priority	Change the Java priority of one worker thread.
GET	/api/status	Read the latest queue and worker-state snapshot.

Request DTOs use Bean Validation. Queue capacity must be at least 1, the count used when dynamically adding workers must be at least 1, and Java thread priority is limited to the range 1–10.

Thread priority is exposed through the API but is treated only as a scheduler hint. Application correctness does not depend on priority because actual scheduling behavior is determined by the JVM and operating system.

A complete OpenAPI 3.0 definition is stored in `docs/openapi.yaml`.

2.5 Frontend Monitoring Dashboard

The frontend is implemented with React and Vite. It contains three focused components:

- **ControlPanel:** Collects Sender count, Receiver count, and queue capacity. It also exposes buttons for starting/stopping the system, adding workers, and stopping worker groups.
- **QueueStatus:** Displays the current number of queued items and the occupancy percentage using a progress bar.

- **ThreadStatusTable:** Displays Sender and Receiver counts for each logical worker state.

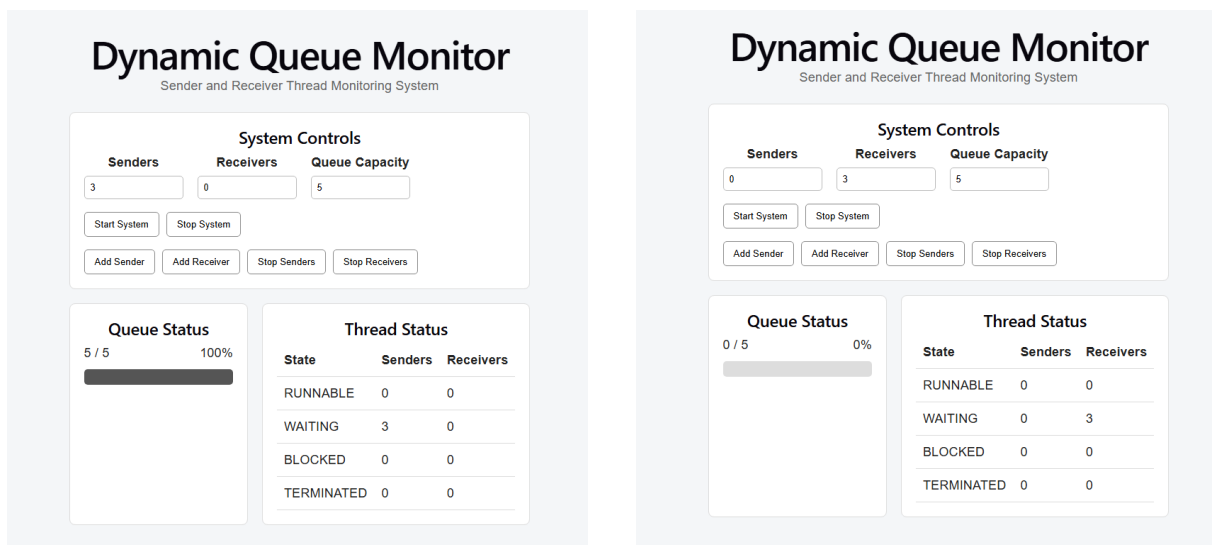
API calls are isolated in `services/api.js`. The main `App` component stores the latest status and polls `/api/status` once per second. The polling interval matches the backend metric collection interval.

REST polling was selected instead of WebSocket because the required monitoring interval is only one second. For this case, polling keeps the implementation smaller and easier to test while still providing sufficiently responsive monitoring.

3. Results

3.1 Functional Scenarios

The application was manually tested with the backend and frontend running together. The following screenshots demonstrate the most important queue and lifecycle scenarios.

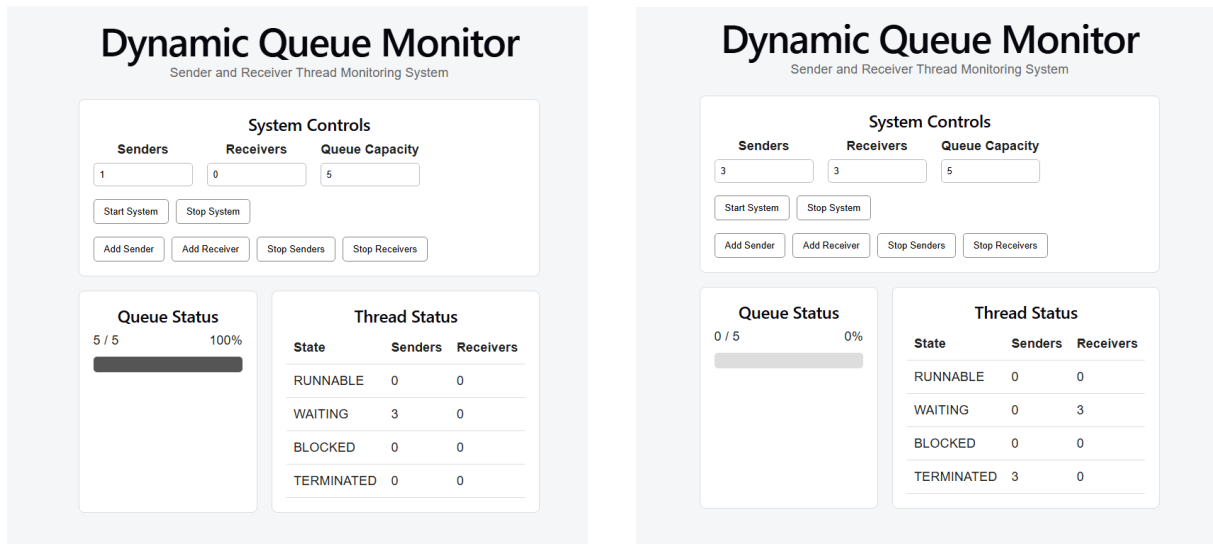


(a) Full queue: 3 Senders, 0 Receivers, capacity 5.

(b) Empty queue: Receivers wait for new data.

Figure 1: Bounded queue waiting behavior.

Figure 1a shows the producer-side boundary condition. With 3 Senders, 0 Receivers, and a capacity of 5, the queue reaches 5/5 items and all Sender workers become `WAITING`. Figure 1b demonstrates the opposite condition: when there is no data to consume, Receiver workers wait on the empty queue.



(a) Workers added dynamically while the system is running.

(b) Sender group stopped while Receivers continue.

Figure 2: Dynamic lifecycle management scenarios.

Figure 2a verifies that new workers can be added without restarting the system. Figure 2b verifies group-level lifecycle management: stopped Sender workers are reported as **TERMINATED**, while remaining Receivers continue until no data remains.

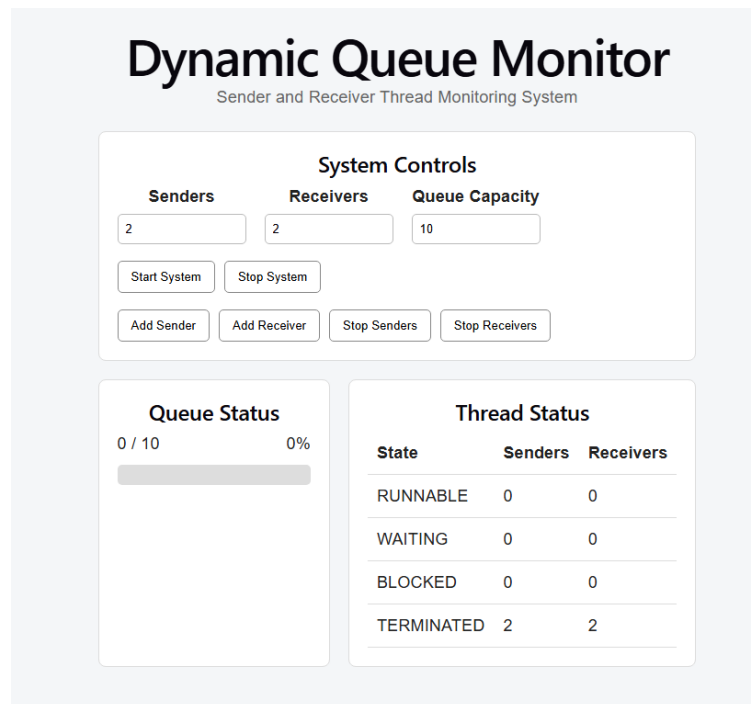


Figure 3: Complete system stop: both worker groups transition to **TERMINATED**.

The complete stop operation shown in Figure 3 terminates both Sender and Receiver groups. Together, these scenarios verify the main runtime controls required by the application.

3.2 Automated Tests

The final backend test suite contains **9 passing tests** with zero failures and zero errors. The tests cover:

- Spring application startup.
- Queue insertion and removal.
- Queue occupancy calculation.
- Invalid queue capacity.
- System startup and status retrieval.
- Dynamic Sender addition.
- Thread-priority API behavior.
- Direct worker priority changes.

Controller behavior is tested with MockMvc. Queue tests exercise the custom bounded queue directly, while the worker-priority test confirms that the underlying Java thread priority is actually updated.

The frontend contains **2 passing component tests** implemented with Vitest and React Testing Library. These tests verify queue size/occupancy rendering and Sender/Receiver state-table rendering. In addition, a Vite production build was executed successfully.

4. Discussion

4.1 Design Decisions

Several design choices were made to keep the solution focused on concurrency rather than infrastructure.

- **Custom bounded queue:** Java provides ready-made blocking queue implementations, but a custom queue based on `ReentrantLock` and `Condition` exposes the synchronization behavior directly. This makes full/empty waiting and lock contention observable.
- **REST polling:** A one-second monitoring interval does not require a persistent bidirectional connection. REST polling is sufficient for the scope and avoids introducing unnecessary WebSocket complexity.

- **In-memory state:** No persistence layer is required because workers and queue contents are runtime-only simulation state.
- **Application-level state model:** The dashboard reports states that describe domain behavior rather than exposing JVM thread states directly.
- **Small component structure:** The React frontend uses focused components and a simple API service instead of introducing a global state-management library.

These decisions keep the project compact while still demonstrating explicit synchronization, thread lifecycle management, REST API design, frontend integration, and automated testing.

4.2 Edge Cases and Concurrency Considerations

Several edge cases were explicitly considered during implementation and testing.

- Starting the system with zero Senders or zero Receivers is valid. This allows full-queue and empty-queue behavior to be tested directly.
- Dynamically adding zero workers is rejected because an add operation must create at least one new worker.
- Workers waiting on `Condition.await()` or sleeping between cycles can be stopped using thread interruption.
- Group stop is asynchronous. Immediately after a stop request, a response may briefly contain the previous state of a worker. The following scheduled metric snapshot reflects the final `TERMINATED` state.
- The logical `BLOCKED` state may be difficult to observe in a one-second snapshot because the critical section around the queue is intentionally short. It is still recorded whenever immediate lock acquisition fails.
- Thread priority is not used as a correctness mechanism. It is exposed as an operational control only, because scheduling behavior is platform-dependent.

The custom queue uses only one lock and two conditions, and queue mutation always occurs while that lock is held. This keeps the synchronization model straightforward and avoids introducing multiple-lock ordering problems.

5. Conclusion

This project successfully implements a multi-channel dynamic queue monitoring system with a Spring Boot backend and React frontend. Multiple Sender and Receiver threads communicate through a shared bounded queue, and the system exposes queue occupancy together with application-level worker states in real time.

The concurrency layer demonstrates explicit Java synchronization using `ReentrantLock`, `Condition`, `volatile`, `ConcurrentHashMap`, and `AtomicInteger`. Dynamic thread creation, group stop, complete system stop, scheduled metrics, and thread-priority management are supported through REST endpoints.

The final application was validated using both automated tests and manual end-to-end scenarios. The backend test suite contains 9 passing tests, the frontend contains 2 passing component tests, and the frontend production build completes successfully. The project also includes setup and API documentation together with an OpenAPI 3.0 specification.

Overall, the solution satisfies the required producer-consumer, monitoring, visualization, lifecycle-management, API, testing, and documentation goals while keeping the architecture intentionally small and focused on the concurrency problem.